

Society Management & Resident Services Platform with Distributed Backend Architecture

CHALLA SREEJA	2520030219
BHIMAVARAPU SAI SARANYA	2520030258
JAWAJI ADVAITH	2520030524

Under the Guidance
of

Dr.Y Subbarayudu

Assitant Professor

Department of Computer Science and Engineering



KONERU LAKSHMAIAH EDUCATION FOUNDATION

GROUP NUMBER: 5

Name of Candidate	Role
CHALLA SREEJA	Backend Architecture & Database Design
BHIMAVARAPU SAI SARANYA	API Development & Microservices
JAWAJI ADVAITH	Frontend Integration & Testing

ABSTRACT

- The Society Management & Resident Services Platform is a web-based system designed to digitize daily
- The Society Management & Resident Services Platform with Distributed Backend Architecture is a centralized digital solution designed to simplify and automate the management of residential communities.
- The platform provides residents, society administrators, security personnel, and service providers with an efficient system for handling daily activities such as resident registration, visitor management, maintenance requests, announcements, facility bookings, complaints, and communication.
- A distributed backend architecture is used to improve scalability, reliability, performance, and secure data management by separating major services into independent modules.
- operations of gated communities and apartment societies.
- It replaces manual registers and WhatsApp-group coordination with a structured platform for maintenance billing, visitor tracking, complaint resolution, and notices.
- The system is built on a distributed backend architecture — separate services for authentication, billing, and notifications communicate through an API gateway, improving scalability and fault isolation over a single monolithic server.
- Core stack: React.js with Tailwind CSS (frontend), Node.js with Express (backend services), and MySQL (relational database), with Redis for caching frequent queries.

INTRODUCTION

- Residential societies still rely on registers, notice boards, and phone calls for tasks like maintenance payments, visitor entry, and complaints.
- This causes delayed payments, lost complaint records, and no visibility for residents into society finances or ongoing issues.
- Our platform gives residents, security staff, and the management committee a single, role-based system to handle these tasks online.
- Instead of one large application handling everything, the backend is split into independent services (auth, billing, visitor, notifications) — a distributed architecture that keeps the system responsive as more societies and residents are added.

EXISTING SYSTEM

- Manual registers at the security gate for visitor entry — slow, error-prone, and not searchable.
- Maintenance fees collected via cash or informal UPI transfers with no automated tracking or receipts.
- Complaints raised verbally or on WhatsApp groups, with no ticketing, status tracking, or accountability.
- Notices pinned on physical boards — residents miss updates if they are travelling or don't check the board.
- Existing society-management apps (MyGate, NoBrokerHood) are closed, subscription-based, and not customizable for a specific society's needs.

PROBLEM

- No centralized system for tracking who has paid maintenance and who hasn't, leading to revenue leakage for the association.
- Visitor entries are not logged digitally, creating a security gap and no audit trail during incidents.
- Complaints get lost between residents, security, and the maintenance committee with no single source of truth.
- A single monolithic backend (if built naively) becomes a bottleneck — one slow module (e.g. bulk notification sending) can slow down unrelated features like login or payments.
- Existing commercial platforms are expensive, not source-owned by the society, and lock the community's data into a third-party vendor.

SOLUTION(Contribution)

- A full-stack, self-hosted platform with a React + Tailwind CSS frontend and a Node.js/Express backend split into independent services rather than one large application.
- Distributed backend design: separate services for Authentication, Billing, Visitor Management, and Notifications, each with its own responsibility, connected through a central API Gateway.
- MySQL as the primary relational database (3NF-normalized schema, indexed for fast lookups), with Redis used to cache frequently-read data such as dashboards and notice feeds.
- Role-based access control (Resident / Security / Admin) so each user only sees what's relevant to them, backed by JWT-based authentication.
- Async communication between services (e.g. a payment event triggering a notification) handled through a message queue, so one slow service never blocks another.

Objectives

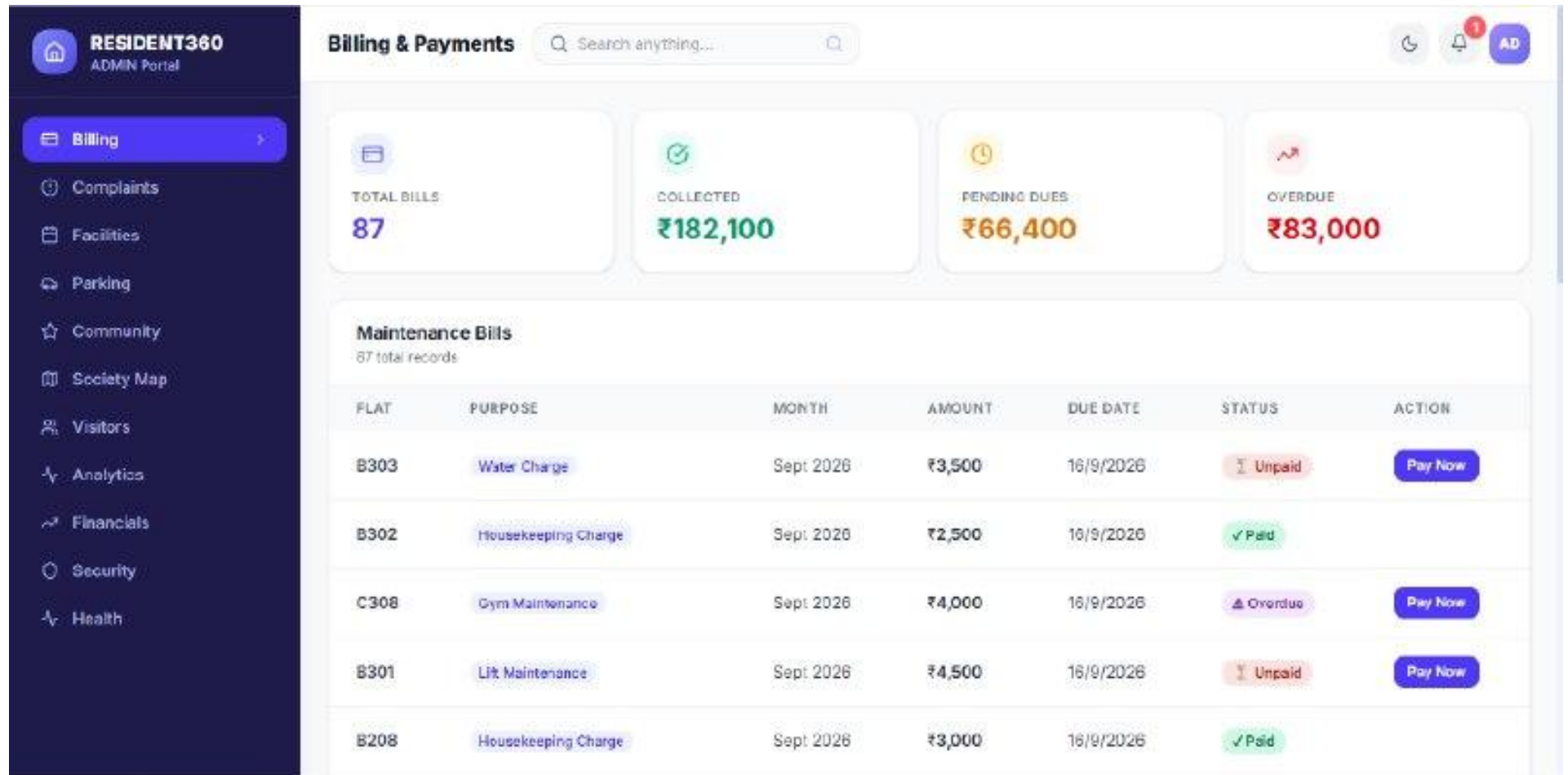
- Digitize maintenance billing with automated invoice generation, payment tracking, and due-date reminders.
- Build a digital visitor log that security staff can update in real time, searchable by resident, date, or visitor name.
- Provide a complaint-ticketing system with status tracking (Open / In Progress / Resolved) and committee accountability.
- Deliver notices and announcements instantly to all residents through the platform instead of a physical board.
- Design the backend as independently deployable services, so the system can scale (e.g. more societies, more residents) without redesigning the whole application.

Modules

- Authentication & Role Management — resident, security, and admin logins with JWT-based sessions and role-based permissions.
- Maintenance & Billing Service — generates monthly invoices, tracks payment status, and stores payment history per flat.
- Visitor Management Service — digital visitor entry/exit logs, pre-approved visitor passes, and searchable records.
- Complaint & Ticketing Service — residents raise complaints, committee assigns and resolves them, with status updates.
- Notice & Notification Service — admin broadcasts announcements; delivered to residents via the dashboard and push/email alerts.
- Admin Dashboard — society-wide view of dues, open complaints, and visitor activity for the management committee.

Proposed System - App

Admin portal :



The screenshot displays the 'Billing & Payments' section of the RESIDENT360 Admin Portal. The left sidebar contains navigation links: Billing (selected), Complaints, Facilities, Parking, Community, Society Map, Visitors, Analytics, Financials, Security, and Health. The main content area features a search bar and four summary cards: Total Bills (87), Collected (₹182,100), Pending Dues (₹66,400), and Overdue (₹83,000). Below these is a 'Maintenance Bills' table with 67 total records. The table lists bills for various flats, including B303, B302, C308, B301, and B208, with details on purpose, month, amount, due date, and status (Unpaid, Paid, or Overdue). Each row includes a 'Pay Now' button.

FLAT	PURPOSE	MONTH	AMOUNT	DUE DATE	STATUS	ACTION
B303	Water Charge	Sept 2026	₹3,500	16/9/2026	Unpaid	Pay Now
B302	Housekeeping Charge	Sept 2026	₹2,500	16/9/2026	✓ Paid	
C308	Gym Maintenance	Sept 2026	₹4,000	16/9/2026	Overdue	Pay Now
B301	Lift Maintenance	Sept 2026	₹4,500	16/9/2026	Unpaid	Pay Now
B208	Housekeeping Charge	Sept 2026	₹3,000	16/9/2026	✓ Paid	

Proposed System - App

Resident portal :

RESIDENT360
RESIDENT Portal

Billing

Complaints

Facilities

Parking

Community

Society Map

Billing & Payments

Q Search anything...

RE

TOTAL BILLS
10

COLLECTED
₹15,700

PENDING DUES
₹10,800

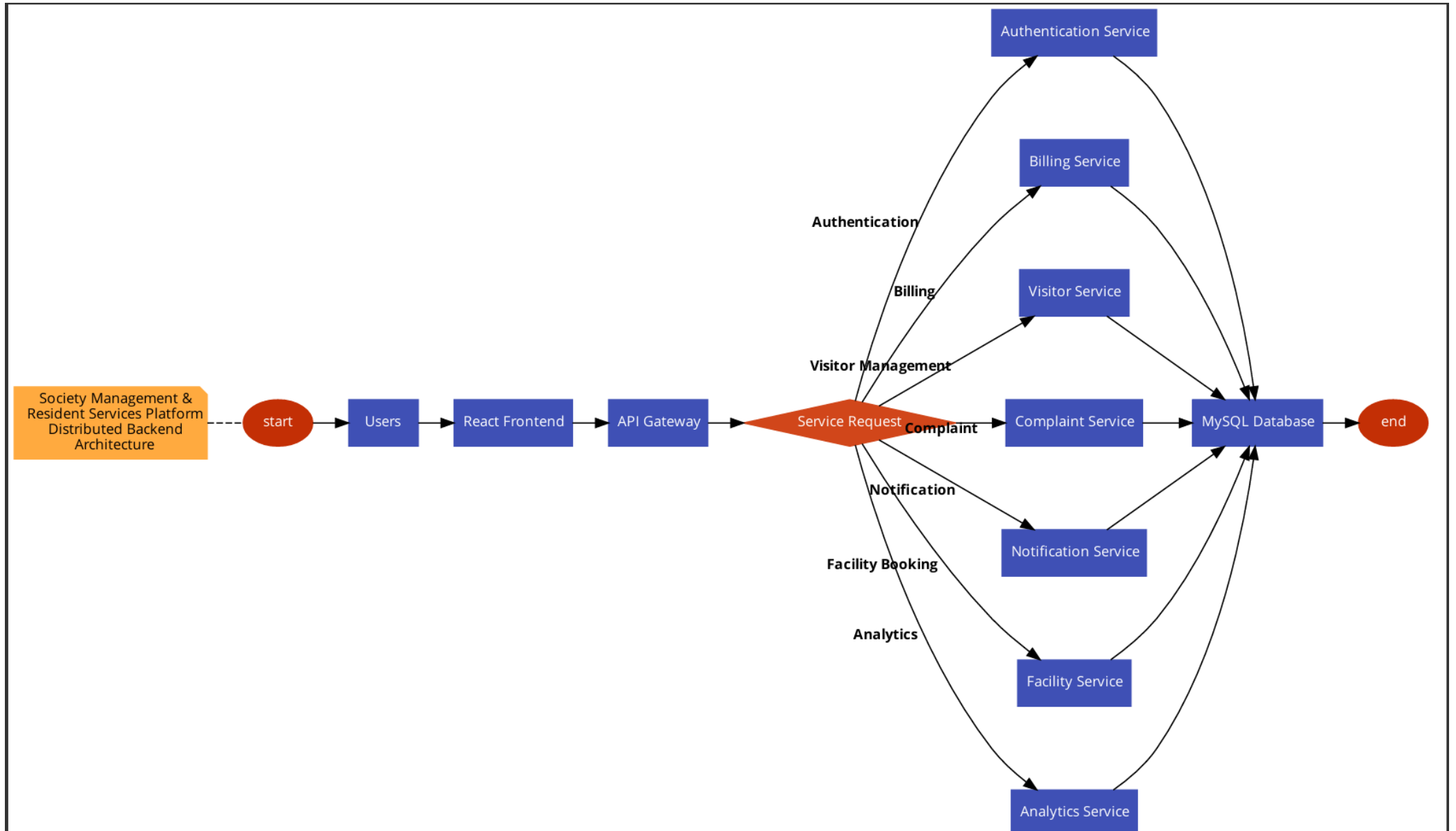
OVERDUE
₹9,500

Maintenance Bills

10 total records

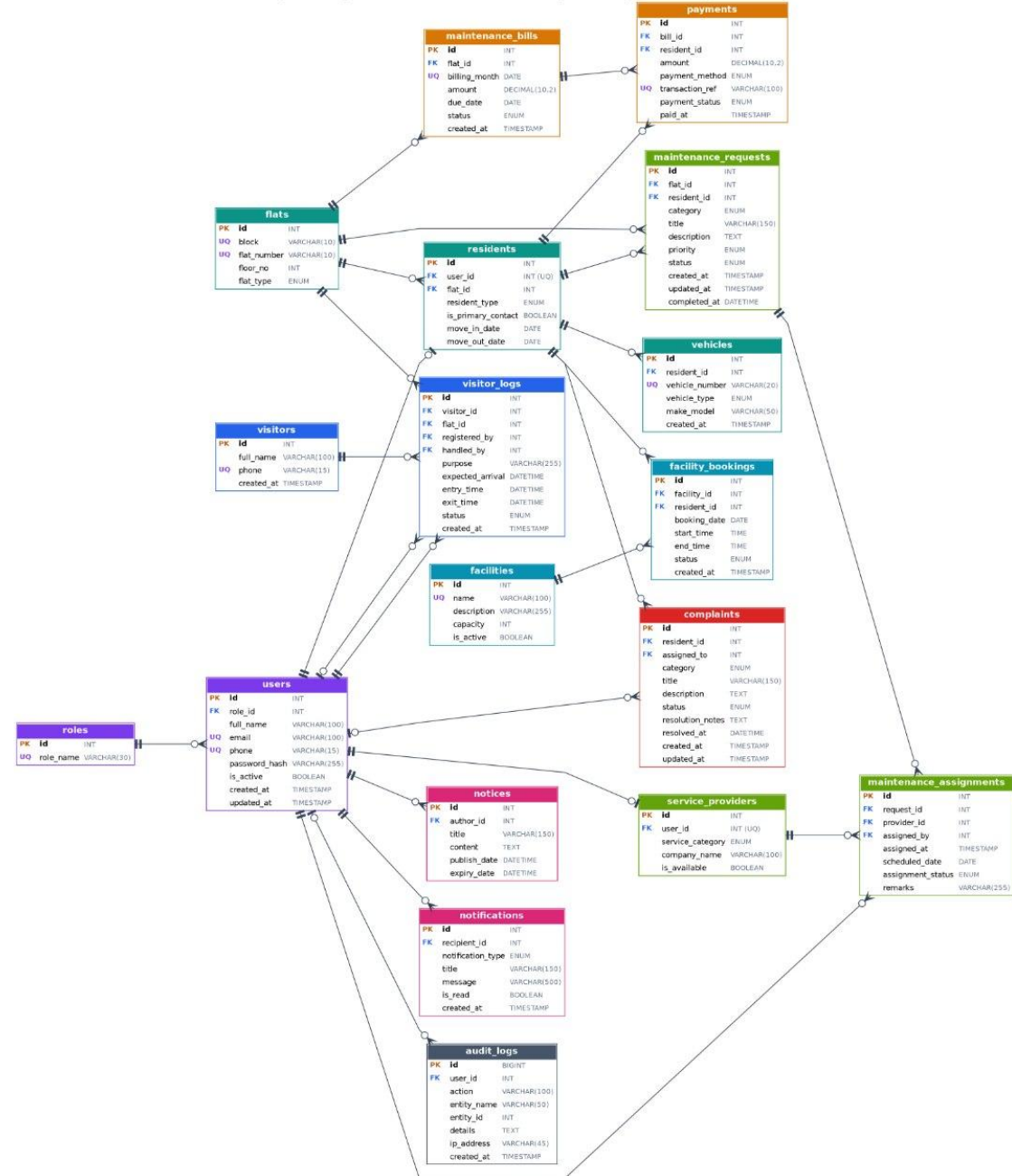
FLAT	PURPOSE	MONTH	AMOUNT	DUE DATE	STATUS	ACTION
A101	CCTV & Security Systems	Sept 2026	₹5,500	16/9/2026	Unpaid	Pay Now
A101	Admin Charge	Aug 2026	₹5,500	16/8/2026	Overdue	Pay Now
A101	Clubhouse Maintenance	Jul 2026	₹3,500	16/7/2026	Paid	
A101	Clubhouse Maintenance	Jun 2024	₹3,200	16/6/2024	Paid	

System Architecture



ER Diagram

Society Management Platform - ER Diagram (MySQL, single society)



SOFTWARE REQUIREMENTS

- Frontend: React.js, Tailwind CSS, Recharts (for the admin dashboard)
- Backend: Node.js, Express.js (structured as independent services)
- Database: MySQL (primary relational store, 3NF-normalized schema)
- Caching: Redis (for dashboard and notice-feed caching)
- Authentication: JWT and bcrypt for password hashing
- Tools: Visual Studio Code, Postman, Git & GitHub, XAMPP/MySQL Workbench
- Operating System: Windows / Linux (server deployment)

CONCLUSION AND FUTURE SCOPE

The platform brings maintenance billing, visitor logging, complaint tracking, and notices onto one system, replacing scattered registers and WhatsApp groups.

Its distributed backend — separate services for auth, billing, visitor, and notifications behind an API gateway — keeps the system responsive and easy to extend as more societies join.

Future scope: a resident mobile app, UPI payment gateway integration, facial-recognition visitor entry, and analytics for the management committee on spending and complaint trends.

THANK YOU